Early experiments such as http://www.cse.scu.edu/~bdezfouli/publication/QUIC-MQTT-COMNET2019.pdf and https://eggert.org/papers/2020-ndss-quic-iot.pdf suggest that QUIC can provide significant savings in an IoT context, but the protocol has not yet been published as a final standard. Although not yet achieving widespread adoption in IoT applications, it's likely that QUIC will become increasingly important over the next few years.

Although Java supports DTLS, it only does so in the form of the low-level `SSLEngine` class, which implements the raw protocol state machine. There is no equivalent of the high-level `SSLSocket` class that is used by normal (TCP-based) TLS, so you must do some of the work yourself. Libraries for higher-level protocols like CoAP will handle much of this for you, but because there are so many protocols used in IoT applications, in the next few sections you'll learn how to manually add DTLS to a UDP-based protocol.

> **NOTE** The code examples in this chapter continue to use Java for consistency. Although Java is a popular choice on more-capable IoT devices and gateways, programming constrained devices is more often performed in C or another language with low-level device support. The advice on secure configuration of DTLS and other protocols in this chapter is applicable to all languages and DTLS libraries. Skip ahead to section 12.1.2 if you are not using Java.

IMPLEMENTING A DTLS CLIENT

To begin a DTLS handshake in Java you first create an `SSLContext` object, which indicates how to authenticate the connection. For a client connection, you initialize the context